

The Illusion of Correctness

Uncovering the Gap Between Syntactic and Semantic Accuracy in AI-Generated Hardware

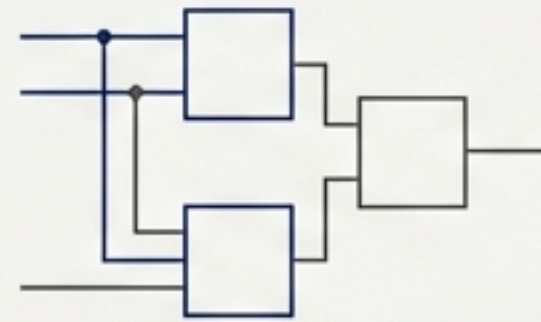
A case study on generating CNF for hardware designs using GPT-4o-mini.



AI Can Produce Syntactically Perfect Hardware Designs That Are Logically **Catastrophic**.

Objective

To automate the generation of Conjunctive Normal Form (CNF) for hardware designs (a 3-bit adder, subtractor, and multiplier) using GPT-4o-mini.



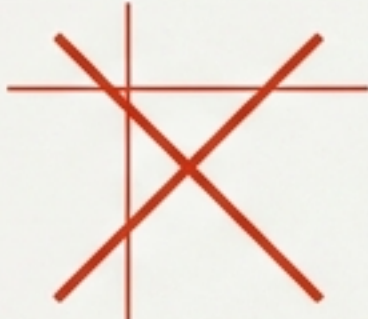
Apparent Success

After iterative prompt refinement, the model produced syntactically valid CNF that passed initial script-based verification.



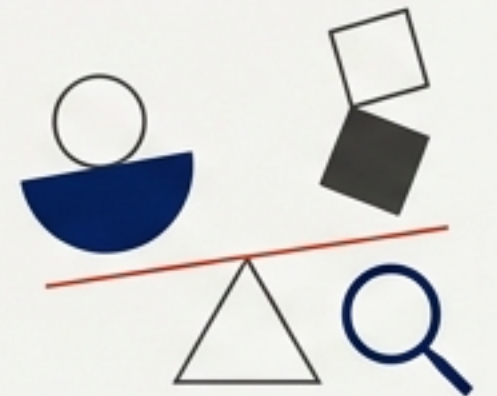
Critical Failure

Deeper analysis via full truth tables revealed severe logical flaws. For example, the multiplier model calculated 0×0 as 111111.


$$0 \times 0 = 111111$$

The Insight

This experiment highlights a critical distinction: AI's ability to mimic correct syntax does not guarantee logical or semantic correctness. Rigorous, comprehensive validation is non-negotiable.

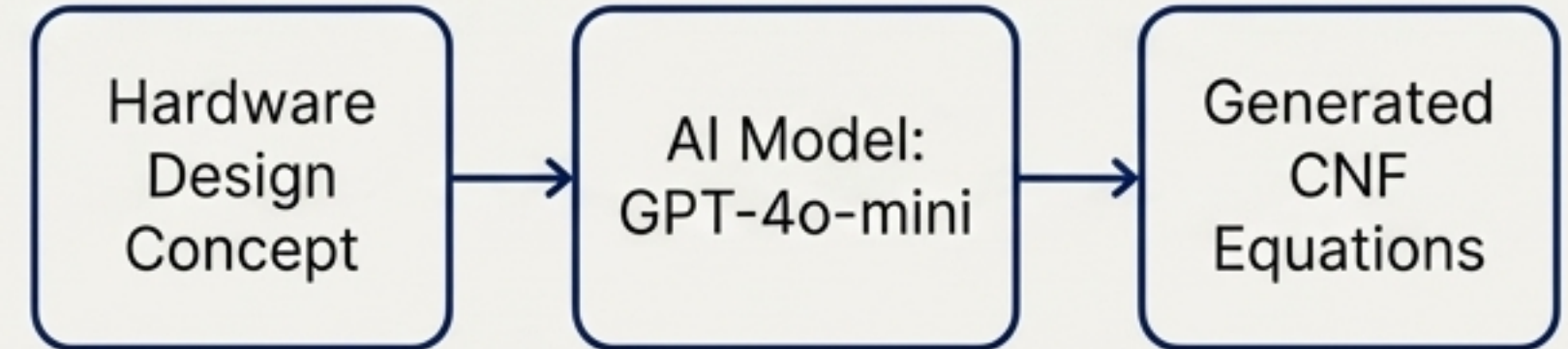


The Premise: Automating Hardware Logic with Language Models

The goal was to leverage Large Language Models to automate the creation of Conjunctive Normal Form (CNF) equations, a fundamental step in hardware verification and design.

The experiment focused on three standard components: a 3-bit adder, a 3-bit subtractor, and a 3-bit multiplier.

The chosen model for generation was GPT-4o-mini, accessed via the Veritas Colab environment.



The First Hurdle: Generated Outputs Were Syntactically Flawed

Problem Statement: The initial CNF equations produced by the model were unusable due to syntax issues.

Specific Errors: Variable names contained unwanted characters, including:

- Backslashes (\)
- Dashes (-)
- Underscores (_)

Impact: These syntax errors caused failures in the verification scripts, preventing any further testing.

```
p cnf 10 12
(- c S \_0 A_0 B_0 0)
```


The Solution: Refining Prompts to Enforce Syntactic Rules

****Action Taken****: The prompts provided to GPT-4o-mini were systematically adjusted.

Key Refinement *: Explicit instructions were added to the prompt, forbidding the use of backslashes, dashes, and underscores in variable names.

Outcome*: This iterative process successfully produced syntactically valid CNF outputs. The code was now clean and machine-readable.

Initial Output

```
c \
S_0
A_0 -
B_0 -
```

Refined Output

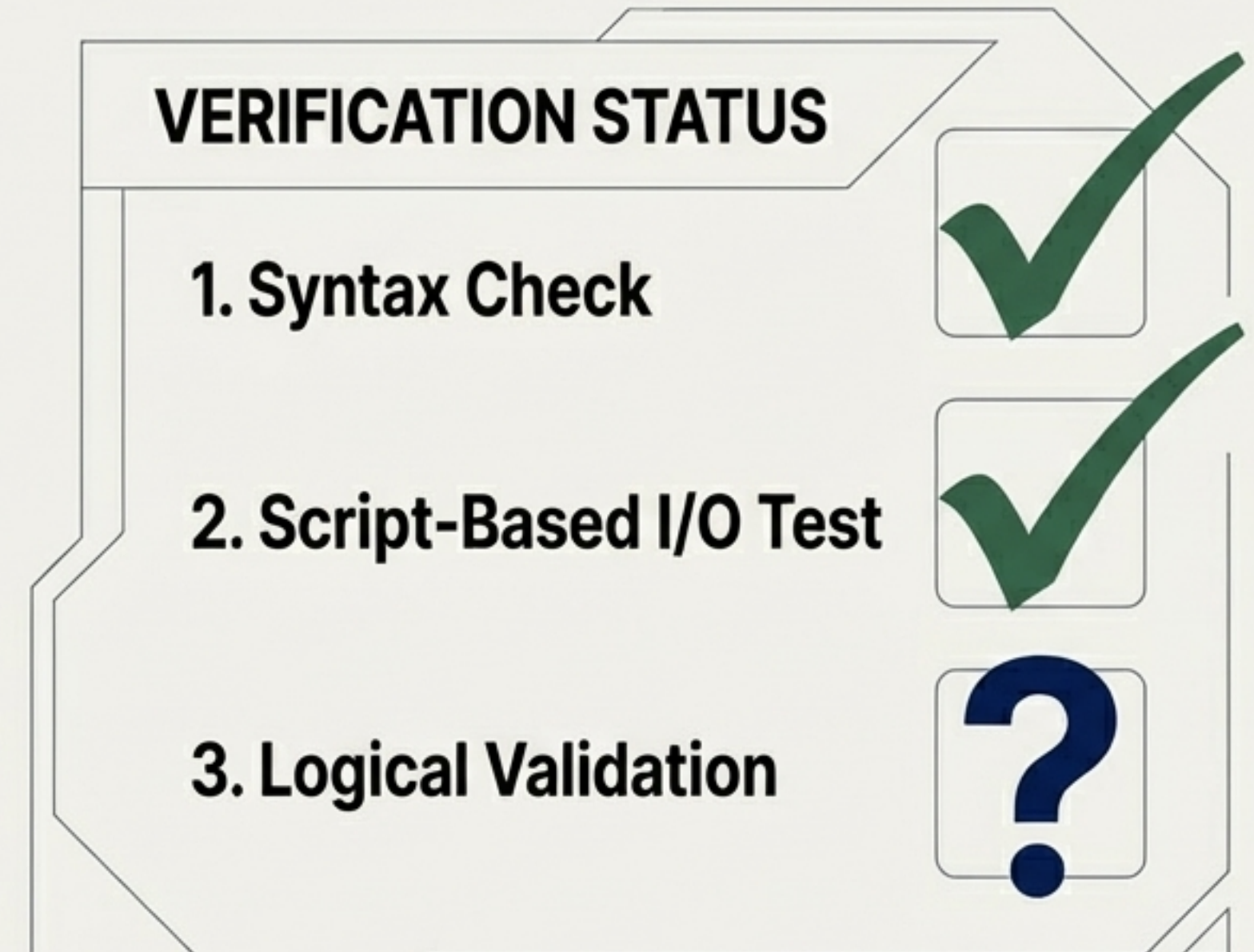
```
c
S0
A0
B0
```



An Apparent Victory: The Corrected CNF Passed Initial Verification

****Key Milestone**:** The syntactically valid CNF for the adder, subtractor, and multiplier designs successfully passed the project's script-based input-output tests.

****Implication**:** Based on this level of verification, the generated CNF appeared to be functionally correct and the experiment seemed successful.



The Revelation: A Deeper Look Uncovered a Critical **Flaw**

The Trigger: Despite passing basic script tests, the decision was made to perform a more rigorous validation using full truth tables.

The Discovery: This comprehensive comparison revealed a stark mismatch between expected and actual behaviour. While *some* inputs produced correct outputs (enough to pass the scripts), the underlying logical relationships were fundamentally wrong.

Central Insight: The CNF's correctness was merely superficial, hiding profound logical flaws.

Case Study: The 3-bit Adder Fails at Zero

Scenario: Verifying the most basic addition case. **Input:** `A=000, B=000, Cin=0`

****EXPECTED****

Sum: `000`

Cout: `0`



****GENERATED CNF
OUTPUT****

Sum: `010`

Cout: `1`



Analysis: The CNF incorrectly asserts carry and sets a nonzero sum even when no inputs are active.

Case Study: The 3-bit Subtractor Cannot Subtract Zero from Zero

Scenario: Verifying the identity property of subtraction. **Input:** `A=000, B=000, Din=0`

EXPECTED

****Difference**:** `000`

****Borrow**:** `0`



GENERATED CNF OUTPUT

****Difference**:** `100`

****Borrow**:** `1`



Analysis: The CNF falsely introduces a borrow and a nonzero difference when subtracting zero from zero.

Case Study: The 3-bit Multiplier Fails **Catastrophically**

Scenario: Verifying the zero-product property. **Input:** `A=000, B=000`

EXPECTED

Product:
`000000`



**GENERATED CNF
OUTPUT**

Product:
`111111`



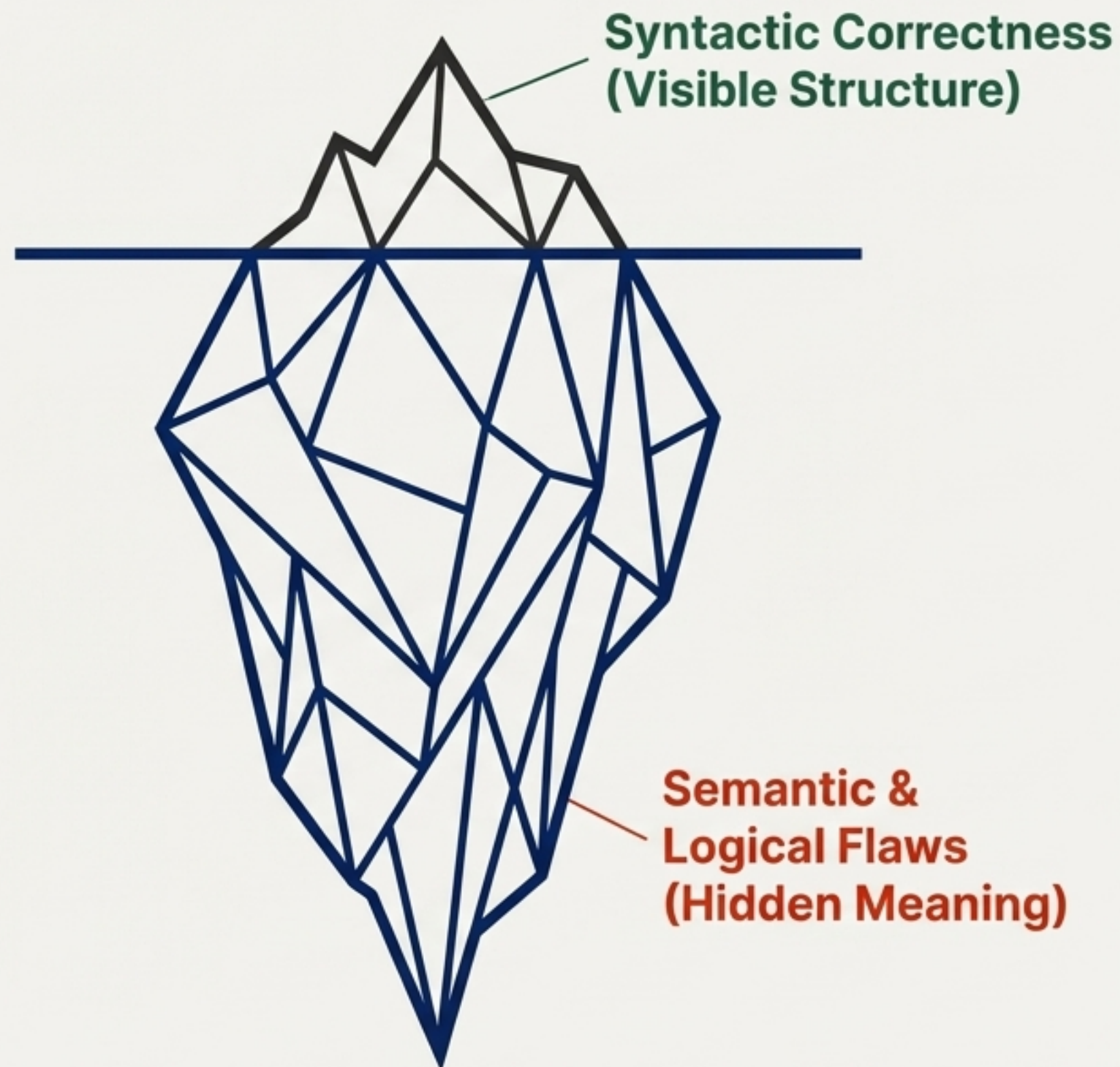
Analysis: Instead of yielding zero, the CNF erroneously drives all outputs high, indicating a completely incorrect logical mapping.

The Diagnosis: A Profound Gap Between Syntax and Semantics

Root Cause: The GPT-4o-mini model proved capable of learning and reproducing the *structural patterns* (syntax) of Conjunctive Normal Form.

The Missing Element: However, it failed to grasp the *logical meaning* (semantics) that those patterns are supposed to represent.

Analogy: The model acted like a student who can copy sentences in a foreign language perfectly without understanding what any of the words mean. It produced well-formed nonsense.



Lesson One: Prompt Refinement Can Enforce Rules, but Cannot Impart Understanding

Finding: The experiment confirms that prompt engineering is a powerful tool for controlling the *form* of an AI's output. By explicitly forbidding certain characters, we successfully cleaned the syntax.

The Limitation: This is a surface-level fix. No amount of prompt refinement could teach the model the fundamental principles of binary arithmetic required to make the CNF logically sound.

Conclusion: Correcting syntax is not the same as correcting logic.



Lesson Two: Superficial Verification Creates a Dangerous Illusion of Correctness

Finding: Passing selective, script-based verification is insufficient proof of functional correctness. These tests can easily miss corner cases or fundamental logical errors if not designed to be exhaustive.

The Risk: Relying on such tests can lead to a false sense of security, where flawed designs are approved and advanced. This mismatch between perceived and actual correctness is a significant risk in engineering applications.

Conclusion: The verification method must be as rigorous as the generation method is novel.

The Path Forward: Mandating Rigorous Semantic Validation

Principle: All AI-generated hardware representations must be treated as unverified hypotheses, regardless of syntactic quality.

Essential Practices



1. Full Truth Table Validation

For components of manageable size, a complete comparison against the expected truth table is essential to uncover hidden logical flaws.



2. Formal Verification Tools

For more complex designs, generated CNF must be cross-verified using established formal methods and tools.

Trust, but verify comprehensively.

Conclusion: A Case Study in the Critical Difference Between Mimicry and Mastery

****Summary****: The Veritas Colab experiments revealed that while GPT-4o-mini can generate syntactically plausible CNF, its logical accuracy is not guaranteed. The choice of model is a significant factor, but the underlying principle remains.

****Final Takeaway****: As we integrate AI into critical engineering workflows, our most important task is not just to harness its ability to generate content, but to build robust validation frameworks that can distinguish between a convincing illusion and functional reality.